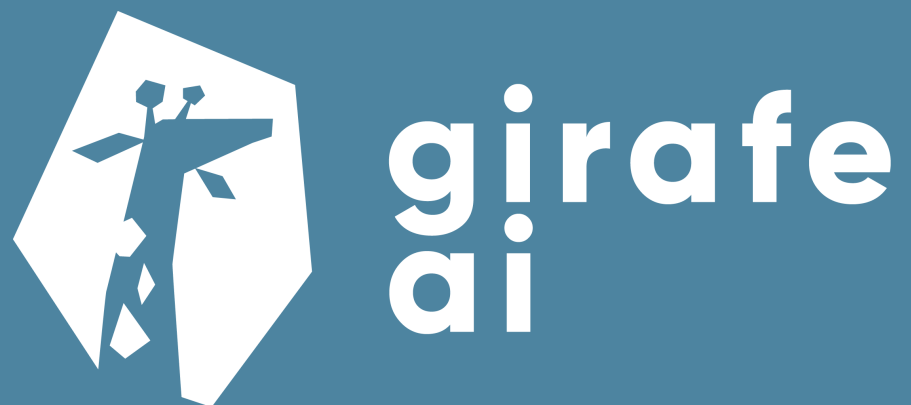


Training speedup

Deploy preparation

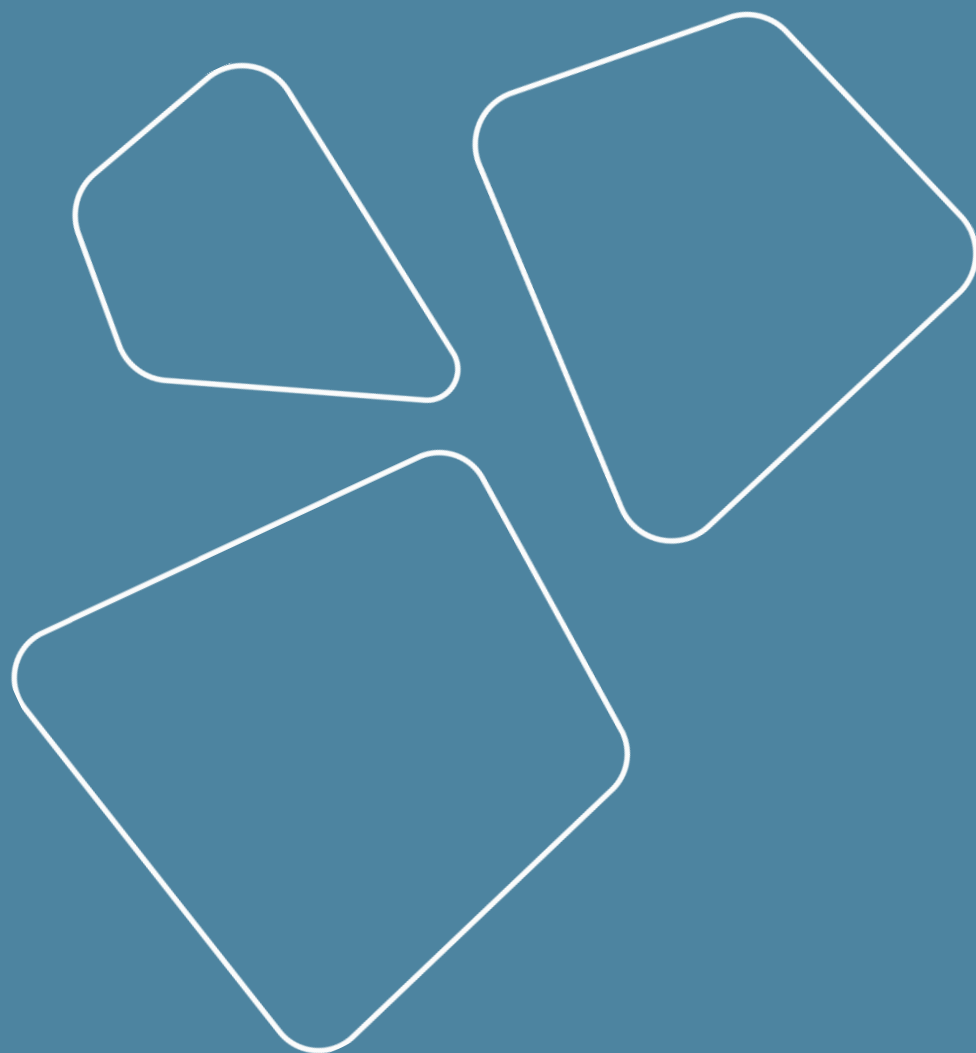
Vladislav Goncharenko

HSE, fall 2025



Outline

- Mixed Precision Training и тензорные ядра
- TensorRT



Mixed Precision

Ускоряем обучение

girafe
ai

01

TLDR

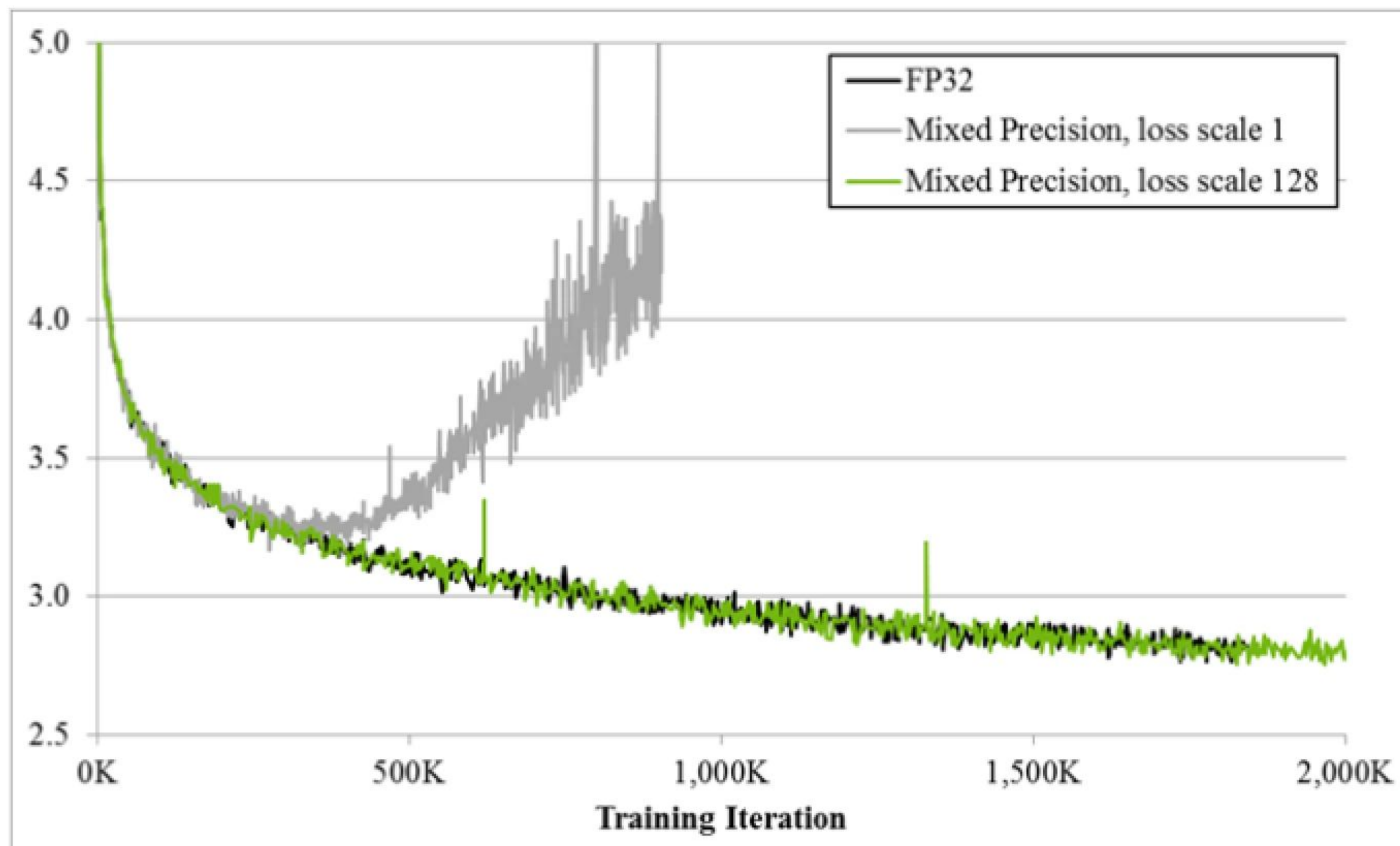
Делаем:

- Заставляем модель использовать FP16 там, где это выгодно
- Добавляем Loss Scale для борьбы с затуханием градиента

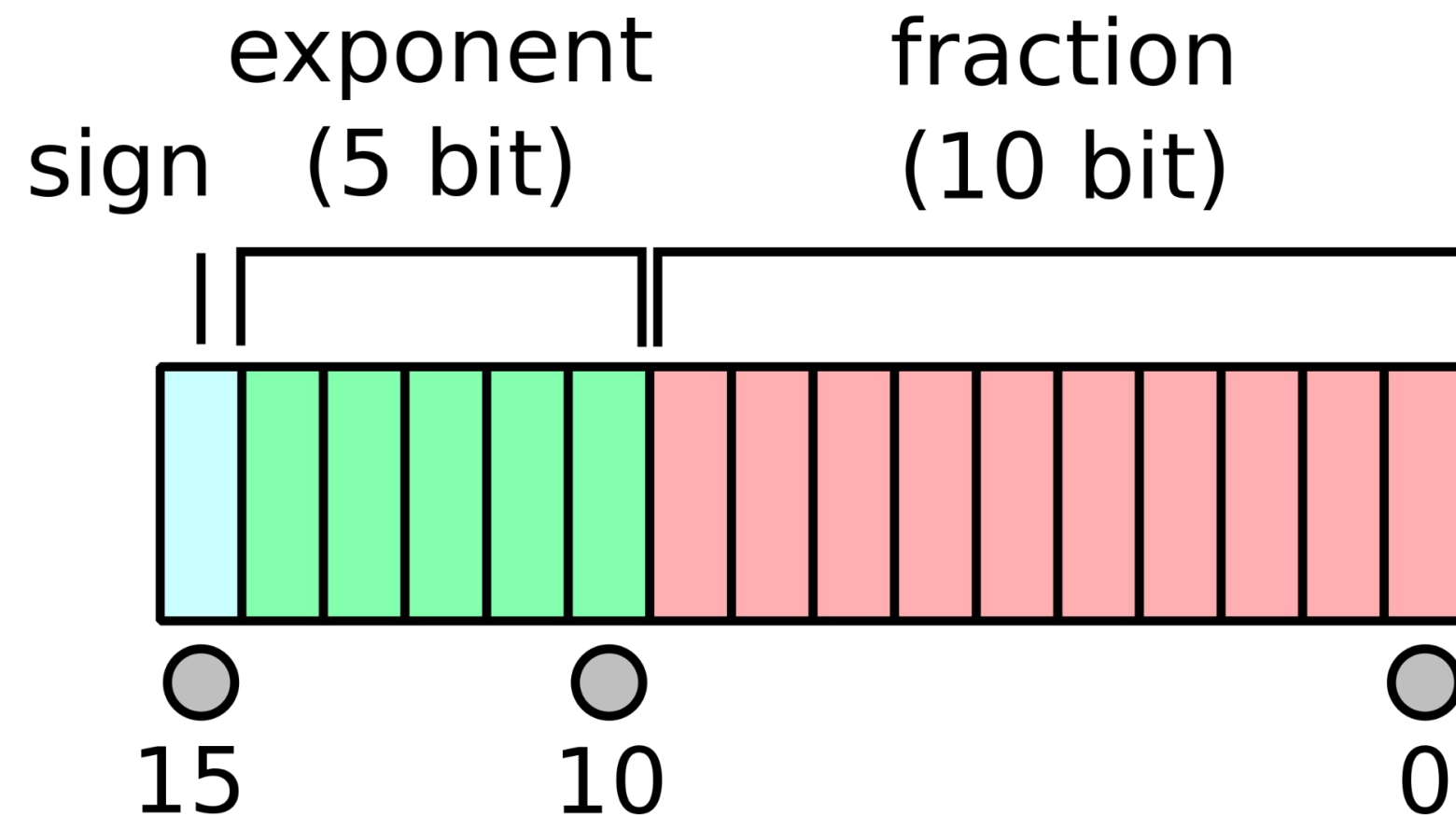
Получаем:

- Более быстрое обучение, ускорение до 8х раз (нет)
- Уменьшение количества необходимой видеопамяти
 - То есть можем использовать батчи большего размера
 - Меньше оверхед на память

Пример

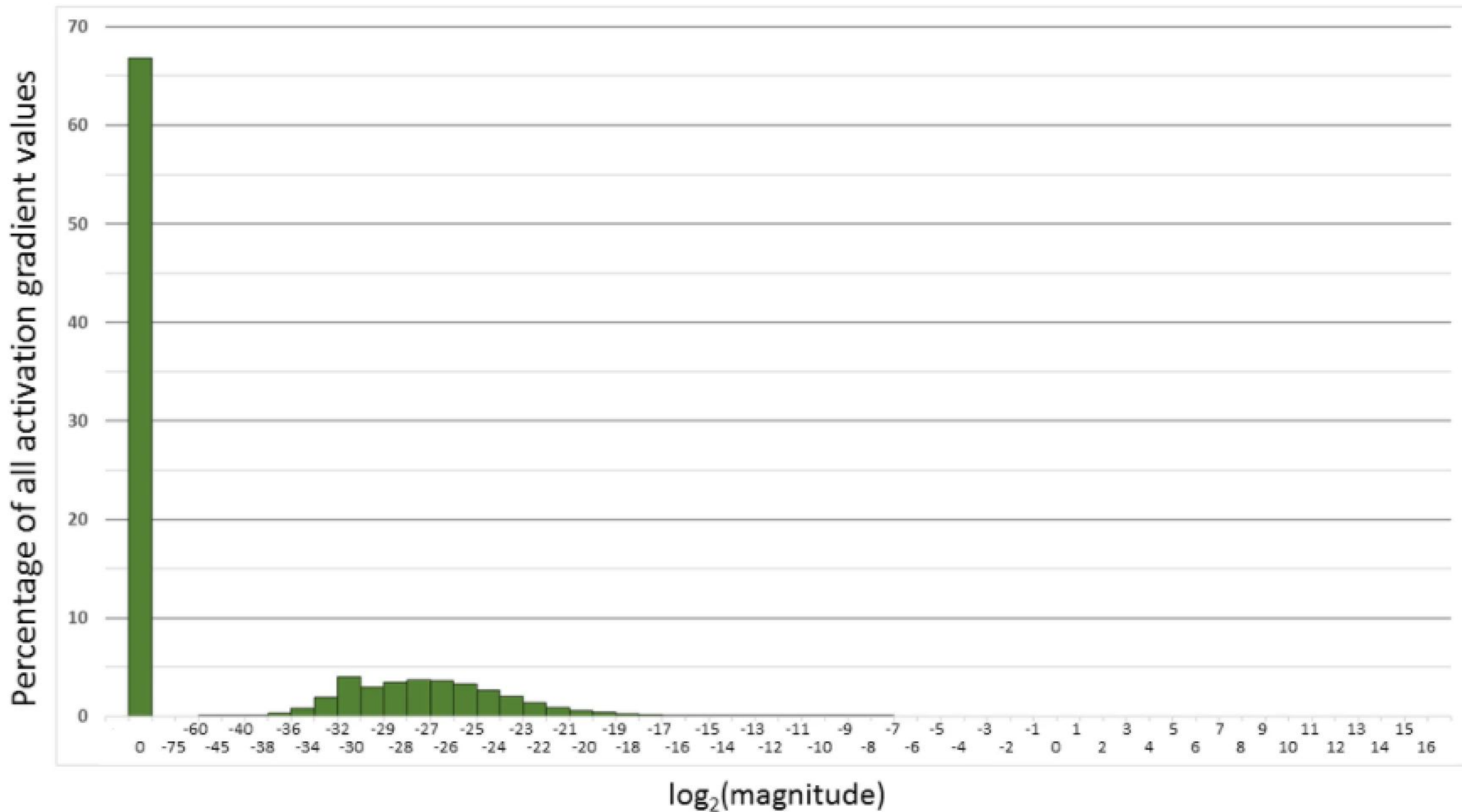


Half Precision

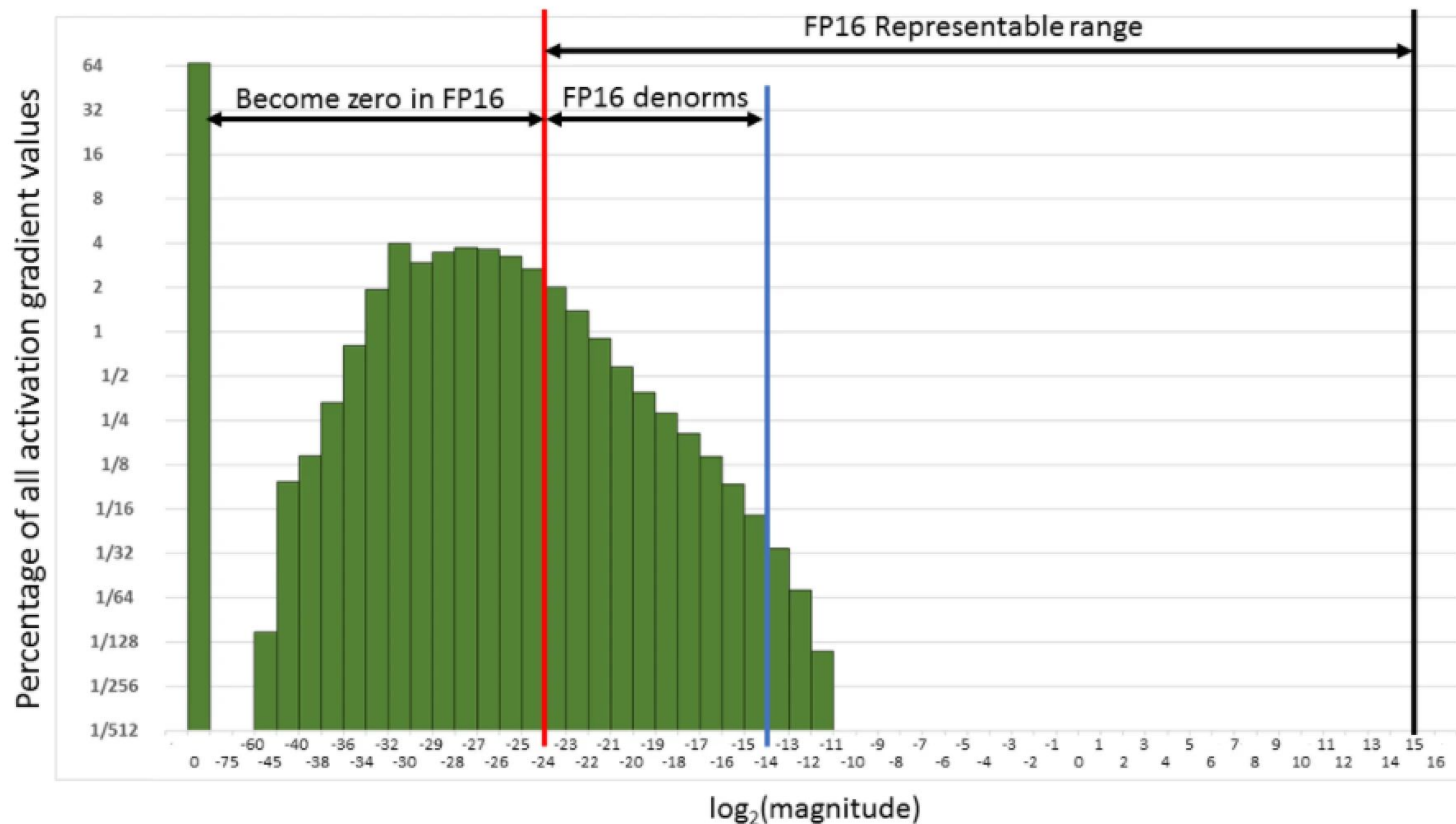


- Normalized:
 - от $2^{(-14)}$ до $2^{(15)}$
- Denormalized:
 - от $2^{(-24)}$ до $2^{(-15)}$
- Максимальное нормализованное:
 - **65504**
- Минимальное нормализованное:
 - **~0.00006**
- Минимальное **де**нормализованное:
 - **6e-8**

Магнитуды градиентов FP32



Магнитуды градиентов FP32



Loss Scaling

1. Maintain a primary copy of weights in FP32.
2. For each iteration:
 - Make an FP16 copy of the weights.
 - Forward propagation (FP16 weights and activations).
 - Multiply the resulting loss with the scaling factor S .
 - Backward propagation (FP16 weights, activations, and their gradients).
 - Multiply the weight gradient with $1/S$.
 - Complete the weight update (including gradient clipping, etc.).

Scaling Factor

Статический подход:

- Выберем S так, что его произведение с максимальным абсолютным значением градиента было < 65504

Динамический подход:

- ???????

Scaling Factor

1. Maintain a primary copy of weights in FP32.
2. Initialize S to a large value.
3. For each iteration:
 - a. Make an FP16 copy of the weights.
 - b. Forward propagation (FP16 weights and activations).
 - c. Multiply the resulting loss with the scaling factor S .
 - d. Backward propagation (FP16 weights, activations, and their gradients).
 - e. If there is an Inf or NaN in weight gradients:
 - i. Reduce S .
 - ii. Skip the weight update and move to the next iteration.
 - f. Multiply the weight gradient with $1/S$.
 - g. Complete the weight update (including gradient clipping, etc.).
 - h. If there hasn't been an Inf or NaN in the last N iterations, increase S .

AMP - Automatic Mixed Precision

Steps:

- Converting the model to use the **float16** data type **where possible**.
- **Keeping float32 master weights** to accumulate per-iteration weight updates.
- Using **loss scaling** to preserve small gradient values.

Тензорные ядра и AMP

1. **Satisfy** Tensor Core shape **constraints**
2. **Increase arithmetic intensity**
3. **Decrease** fraction of work in **non-Tensor Core operations**

1. Satisfy Constraints

Matmul:

1. **FP16** - Gemm`s M, N, K кратны **8**
2. **INT8** - Gemm`s M, N, K кратны **16**

Conv:

1. **FP16** - in_channels, out_channels кратны **8**
2. **INT8** - in_channels, out_channels кратны **16**

1. Satisfy Constraints

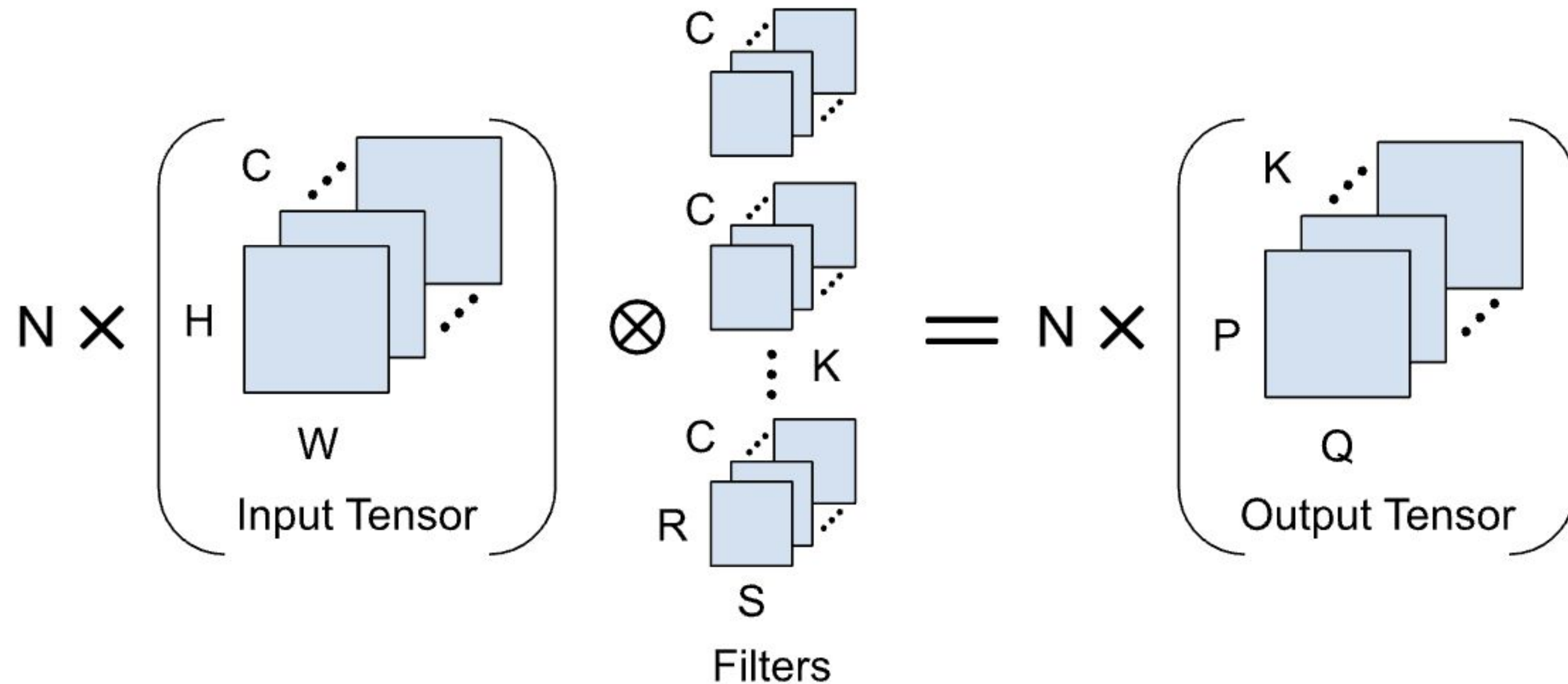
Linear Gemm

Table 1. Mapping of inputs, outputs, and batch size to GEMM parameters M, N, K.

Computation Phase	M	N	K
Forward Propagation	Number of outputs	Batch size	Number of inputs
Activation Gradient	Number of inputs	Batch size	Number of outputs
Weight Gradient	Number of inputs	Number of outputs	Batch size

1. Satisfy Constraints

Conv Gemm



2. Increase Arithmetic Intensity

Model architecture:

- Prefer dense math operations.
 - For example, vanilla convolutions have much higher arithmetic intensity than depth-wise separable convolutions.
- Prefer wider layers when possible.

3. Decrease Non-Tensor Work

Как считается ускорение:

$$1 / (x + ((1-x) / y))$$

- x - доля времени в не Tensor-Core операциях
- y - ускорение от использования тензорных ядер

Например, если *половина* времени обучения тратится на операции в тензорных ядрах, а в этих операциях ускорение равно 5, тогда общее ускорение от использования смешанной точности будет равно $1 / (0.5 + (0.5 / 5)) = 1.67$

Allow / Deny / Infer Lists

- **AllowList:**
 - Convolutions
 - Fully-connected layers
- **DenyList:**
 - Large reductions
 - Cross entropy loss
 - L1 Loss
 - Exponential
- **InferList:**
 - Element-wise operations (add, multiply by a constant)

TyT: https://github.com/NVIDIA/apex/blob/master/apex/amp/lists/functional_overrides.py

Hardware requirements

1. Наличие Тензорных Ядер :)

Примеры ускорений

Model Script	Framework	Data Set	FP32 Accuracy	Mixed Precision Accuracy	FP32 Throughput	Mixed Precision Throughput	Speed-up
<u>BERT Q&A</u>	TensorFlow	SQuAD	90.83 Top 1%	90.99 Top 1%	66.65 sentences/sec	129.16 sentences/sec	1.94
<u>SSD w/RN50</u>	TensorFlow	COCO 2017	0.268 mAP	0.269 mAP	569 images/sec	752 images/sec	1.32
<u>GNMT</u>	PyTorch	WMT16 English to German	24.16 BLEU	24.22 BLEU	314,831 tokens/sec	738,521 tokens/sec	2.35
<u>Neural Collaborative Filter</u>	PyTorch	MovieLens 20M	0.959 HR	0.960 HR	55,004,590 samples/sec	99,332,230 samples/sec	1.81
<u>U-Net Industrial</u>	TensorFlow	DAGM 2007	0.965-0.988	0.960-0.988	445 images/sec	491 images/sec	1.10
<u>ResNet-50 v1.5</u>	MXNet	ImageNet	76.67 Top 1%	76.49 Top 1%	2,957 images/sec	10,263 images/sec	3.47
<u>Tacotron 2 / WaveGlow 1.0</u>	PyTorch	LJ Speech Dataset	0.3629/-6.1087	0.3645/-6.0258	10,843 tok/s 257,687 smp/s	12,742 tok/s 500,375 smp/s	1.18/1.94

Performance Limitations

Пусть:

- T_{mem} - время доступов к памяти
- T_{math} - время выполнения арифметических операций

Суммарное время : $\max(T_{mem}, T_{math})$.

Performance Limitations

Оценим:

- $T_{mem} = \#bytes / BW_{mem}$
- $T_{math} = \#ops / BW_{math}$

, тут $\#bytes$ - количество необходимых байт в памяти, $\#ops$ - количество мат операций, BW_{mem} - Memory Bandwidth, а BW_{math} - Math Bandwidth.

$$T_{math} > T_{mem} \Leftrightarrow \#ops / BW_{math} > \#bytes / BW_{mem} \Leftrightarrow$$

$$\Leftrightarrow \#ops / \#bytes > BW_{math} / BW_{mem}.$$

$\#ops / \#bytes$ -- это и есть арифметическая интенсивность

Performance Limitations

Рассмотрим Nvidia V100:

- $BW_{mem_onchip} = 3.1 \text{ Tb/s}$
- $BW_{mem_offchip} = 900 \text{ GB/s}$
- $BW_{math} = 125 \text{ FP16 TFLOPS}$

=> **арифметическая интенсивность между 40 и 140** (в зависимости от источника данных для операции).

Performance Limitations

Рассмотрим конкретный пример с полносвязанным слоем:

- $\text{in} = 4096, \text{out} = 1024, \text{bs} = 512$
- $\text{FLOP} = 2 * \text{bs} * \text{in} * \text{out} \approx 4 * 10^{**9} \text{ flop}$ (4 GFLOP), (2 так как mul-асс операция)
- $\text{Bytes (weight)} = \text{in} * \text{out} * 2$, (2 байта тк fp16)
- $\text{Bytes (inputs)} = \text{bs} * \text{in} * 2$
- $\text{Bytes (out)} = \text{bs} * \text{out} * 2$
- $\text{Bytes (total)} = \text{in} * \text{out} * 2 + \text{bs} * 2 * (\text{in} + \text{out}) \approx 0.01 * 10^{**9}$

#ops / #bytes = 400 => для nvidia V100 **ТОЧНО** ограничено арифметикой

Performance Limitations

Рассмотрим пример с ReLU идущим за этим линейным слоем:

- $\text{out} = 1024, \text{bs} = 512$
- $\text{FLOP} = \text{bs} * \text{out} \approx 0.5 * 10^{**6} \text{ (MFLOP)}$
- $\text{bytes (input and output)} = \text{bs} * \text{out} * 2 + \text{bs} * \text{out} * 2 \approx 2 * 10^{**6} \text{ (MB)}$

=> **#ops / #bytes = 0.25** => на V100 операция ограничена памятью

Performance Limitations

Попробуем взять тот же самый полносвязанный слой и оценить границу когда операция становится ограничена памятью: in = 4096, out = 1024

- $\#ops / \#bytes = 40$
- $40 = (2 * x * 1024 * 4096) / (1024 * 4096 * 2 + x * 2 * (1024 + 4096))$
- $x = bs \approx 42$

Flash Attention

girafe
ai

02

Flash Attention

- <https://www.youtube.com/watch?v=gBMO1JZav44>
- <https://github.com/Dao-AI-Lab/flash-attention>
- <https://www.datacamp.com/blog/flash-attention>
-

Model storage formats

girafe
ai

02

List of formats

- ONNX
 - crossplatform
- TensorRT by Nvidia
- <https://github.com/ggml-org/ggml/blob/master/docs/gguf.md>
- https://catboost.ai/docs/en/concepts/python-reference_catboost_save_model
- many more...

What not to use: [pickle](#), also [torch.save](#) uses it

Runtimes overview

- ONNX runtime
- Intel OpenVino
 - <https://github.com/openvinotoolkit/openvino>
- Apple CoreML
- Microsoft WinML
- Nvidia TensorRT
- TorchScript
- <https://github.com/Tencent/ncnn>
- etc

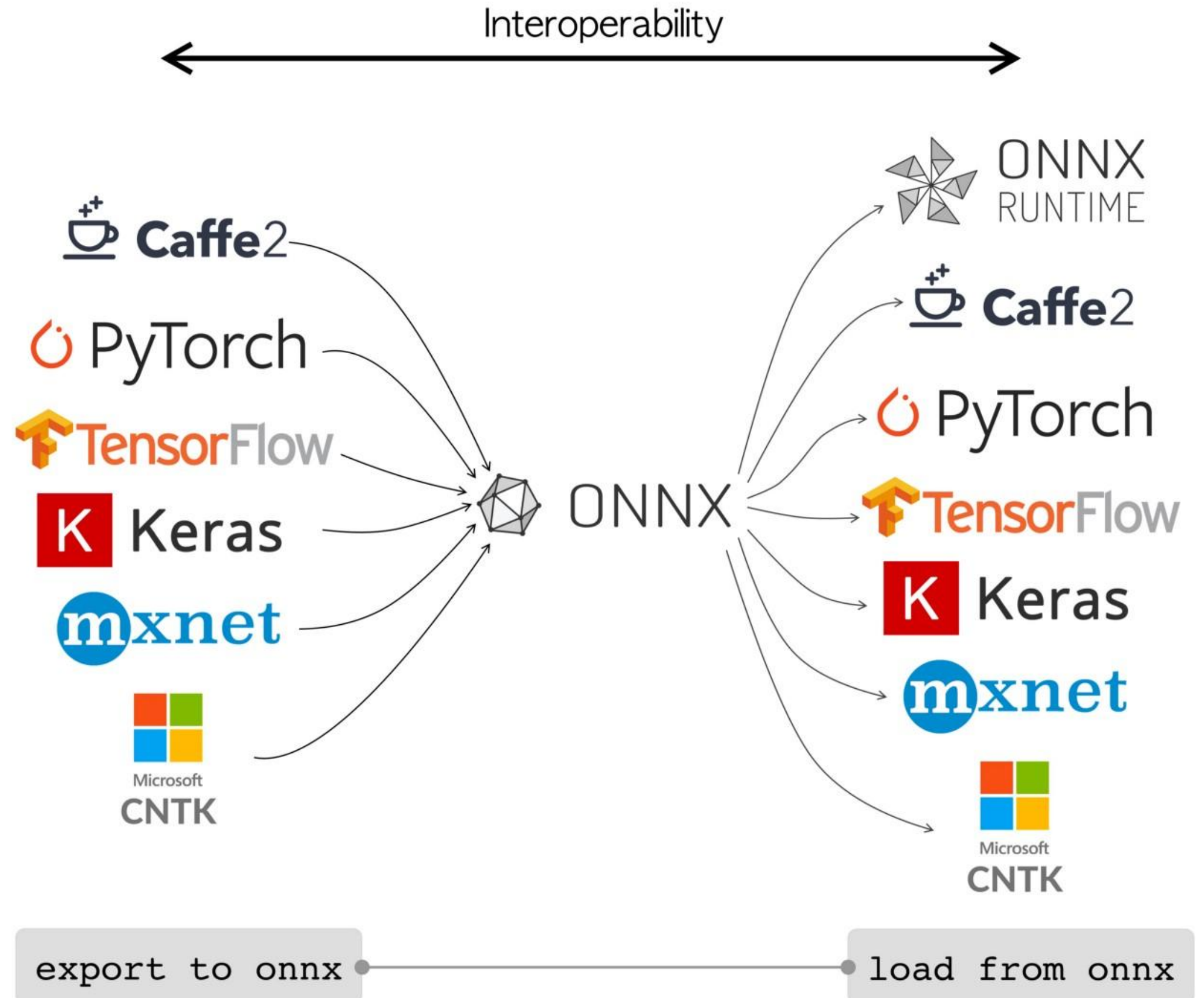
ONNX

girafe
ai

03

ONNX

- <https://www.youtube.com/watch?v=Ij5MoUnLQ0E>
- <https://towardsdatascience.com/onnx-preventing-framework-lock-in-9a798fb34c92/>
- <https://cms-m1.github.io/documentation/inference/onnx.html>
- <https://onnx.ai/onnx/intro/concepts.html>



Protobuf

- <https://protobuf.dev/>

1. Define proto

```
import "person_info.proto";

package persons;

message Person {
    PersonInfo info = 1; // characteristics of the person
    repeated Friend friends = 2; // friends of the person
}
```

2. Compile proto

```
_PERSON = _descriptor.Descriptor(
    name='Person',
    full_name='persons.Person',
    filename=None,
    file=DESCRIPTOR,
    containing_type=None,
    create_key=_descriptor._internal_create_key,
    fields=[
    ...
```

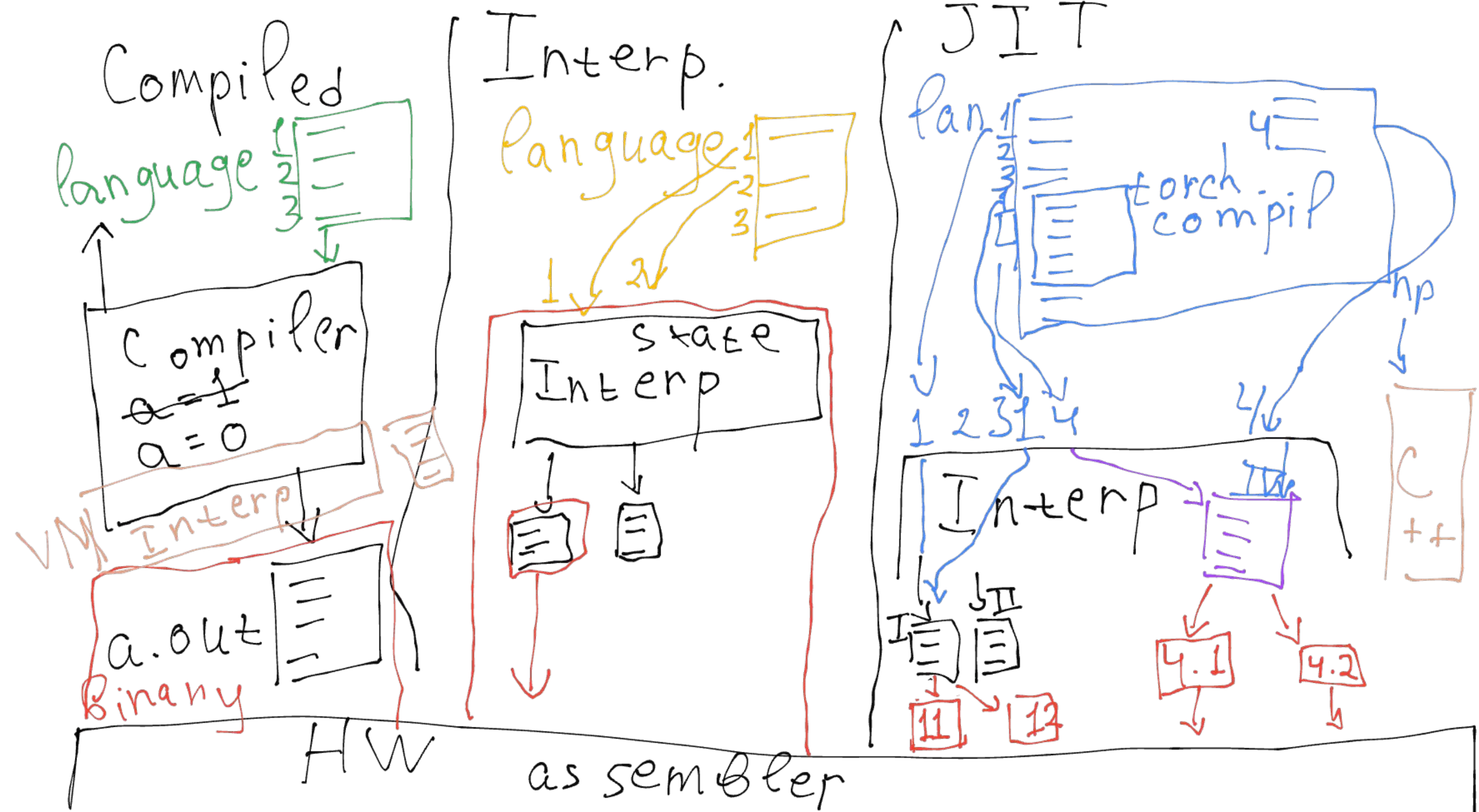
3. Serialize

```
info {
  age: 30
  height: 184
}
friends {
  friendship_duration: 365.1000061035156
  shared_hobbies: "books"
  shared_hobbies: "daydreaming"
  shared_hobbies: "unicorns"
  person {
    info {
      age: 40
      height: 165
    }
  }
}
```

TorchScript and compile

- <https://pytorch.org/docs/stable/jit.html>
- https://pytorch.org/tutorials/beginner/Intro_to_TorchScript_tutorial.html
- https://pytorch.org/tutorials/intermediate/torch_compile_tutorial.html
- <https://pytorch.org/docs/stable/generated/torch.compile.html>
- <https://pytorch.org/docs/stable/torch.compiler.html>
-

JIT



1. GC is introduced in VM based languages
2. Dijkstra and Wirth got rid of goto operator in favour of explicit control flow

TensorRT

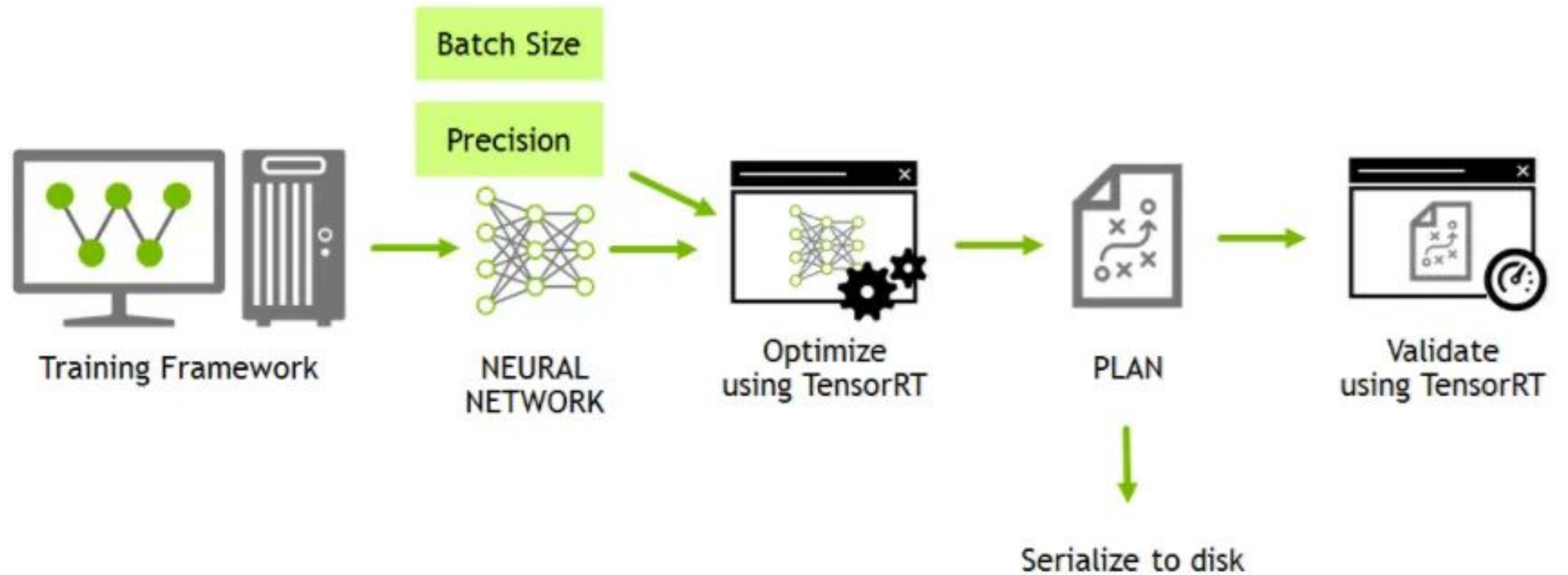
girafe
ai

04

Train vs Infer

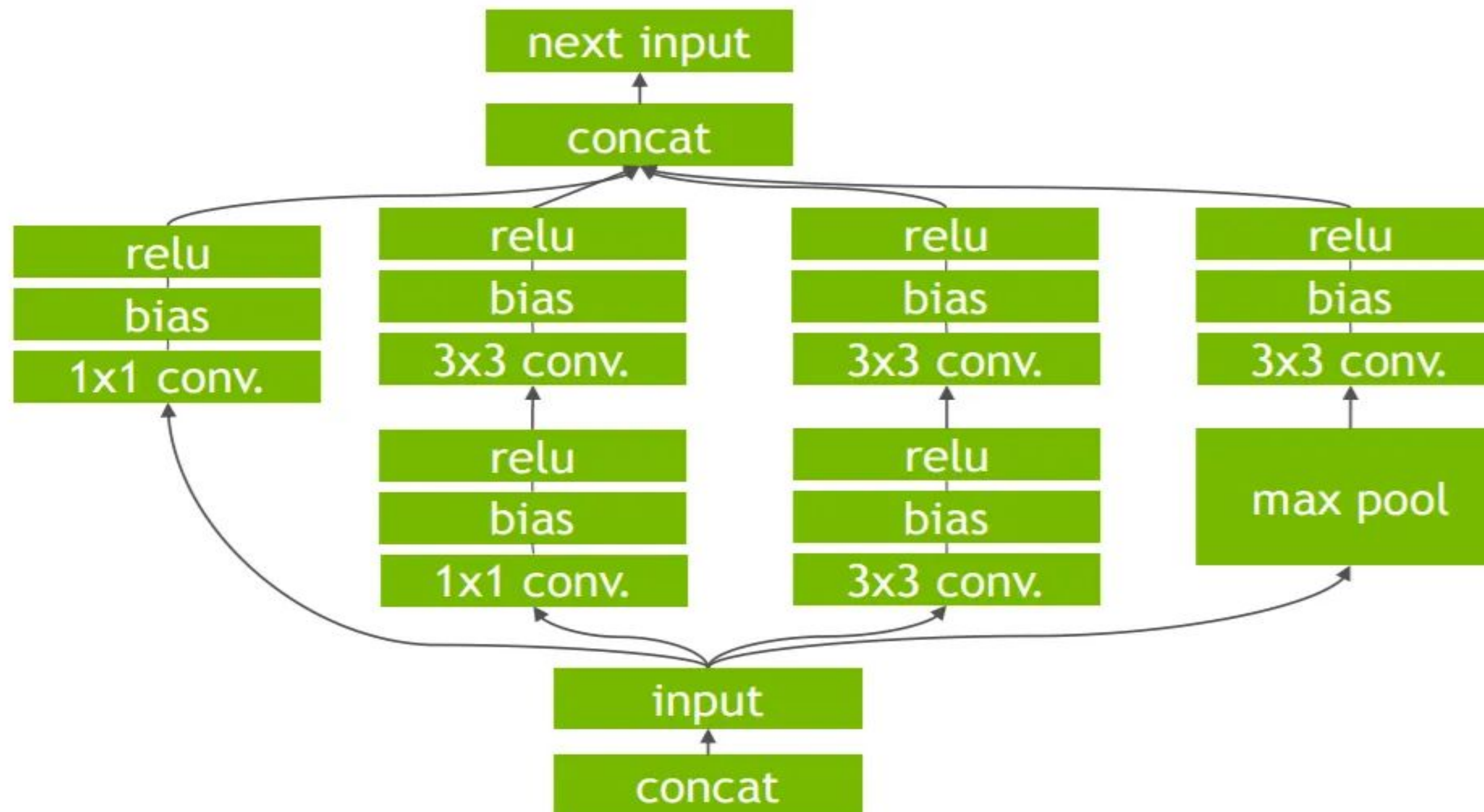
- Железо
 - Много моделей деплоится на CPU
 - Выгодно учить на DGX с A100/V100 с SXM а не PCI-E
 - Но DGX-ы супер прожорливые, особенно в простое
 - 80W idle против 30W idle
 - Обычно не инферим с большим батчем, следовательно не нужно много прожорливой видеопамяти
- Тренировка - оффлайн, инференс - реалтайм
- Нужна высокопроизводительная инфра для обработки запросов
- Как минимум мы не считаем градиенты в инференсе
 - Хотя знаю одно исключение (grad-cam)
- Экономим электричество при инференсе
 - Электричество дорогое
 - В ДЦ вас могут рассчитывать по пику электропотребления
 - У видеокарт нулевой КПД => все электричество переходит в тепло, тепло сложно рассеивать
- Невероятно замороченная оптимизация для деплоя, так как даже 1% к метрике производительности на большом объеме данных может дать прирост в гри-сри-годы или даже столетия
- Большая дисперсия софтины для инференса. Надо заботиться об отказоустойчивости. Нужны балансировщики нагрузки...

TensorRT (aka TRT)



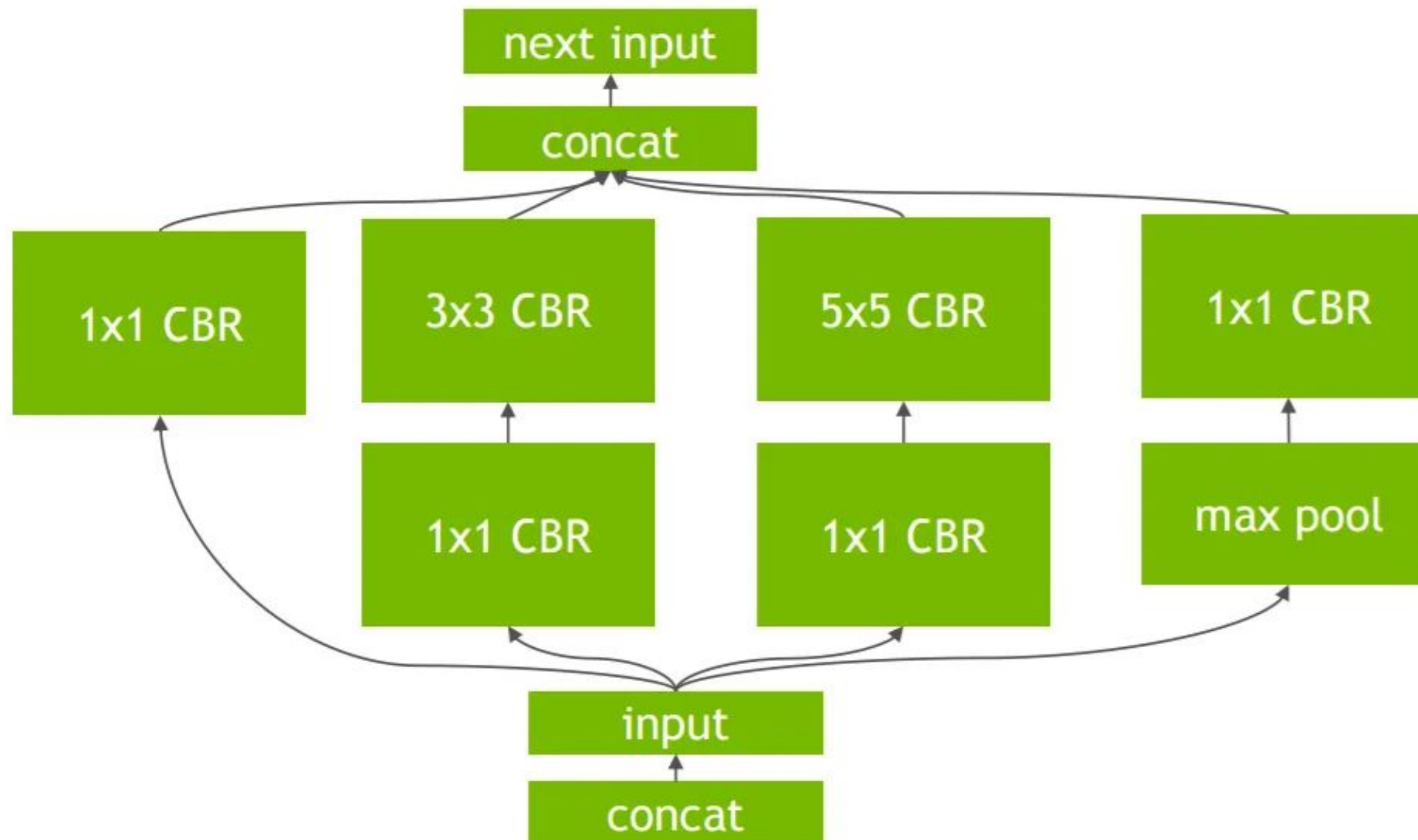
TRT's Layer Fusion

DO

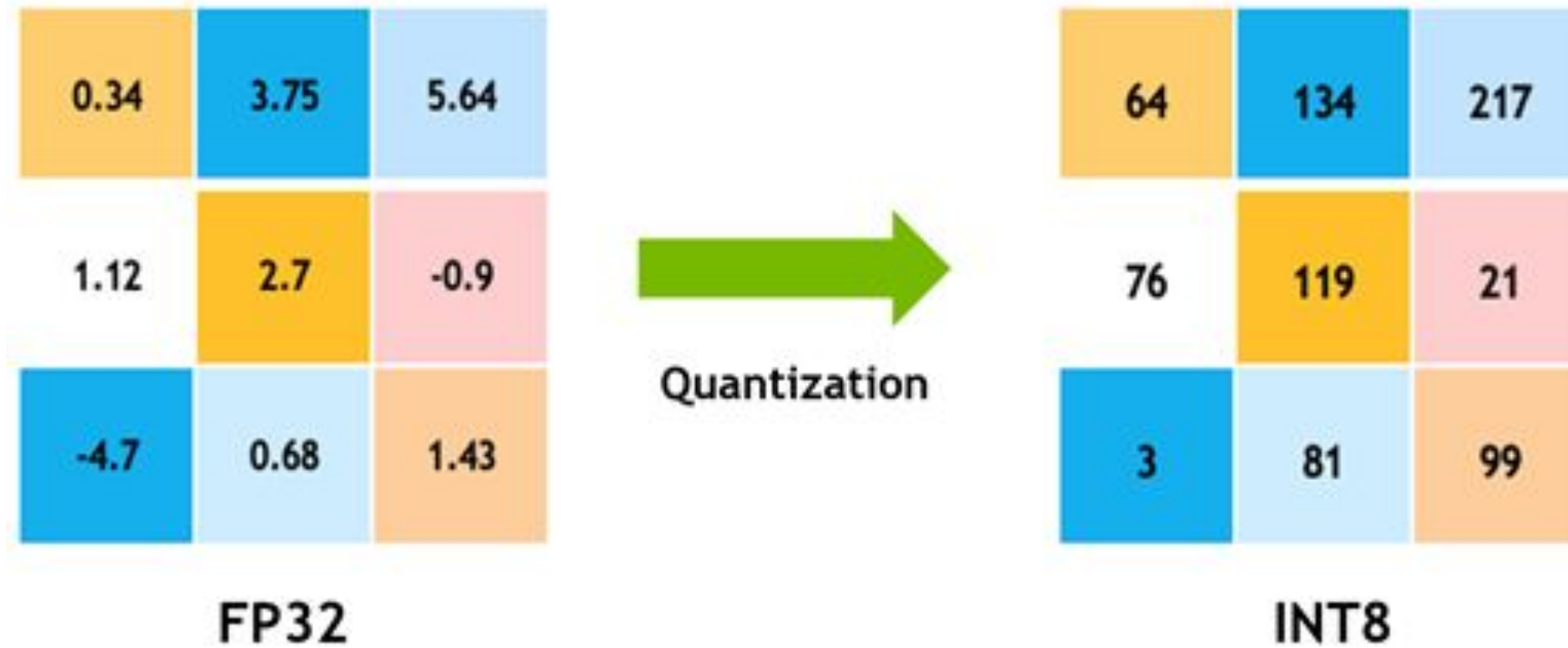


TRT's Layer Fusion

Vertical Fusion



TRT's Static Quantization

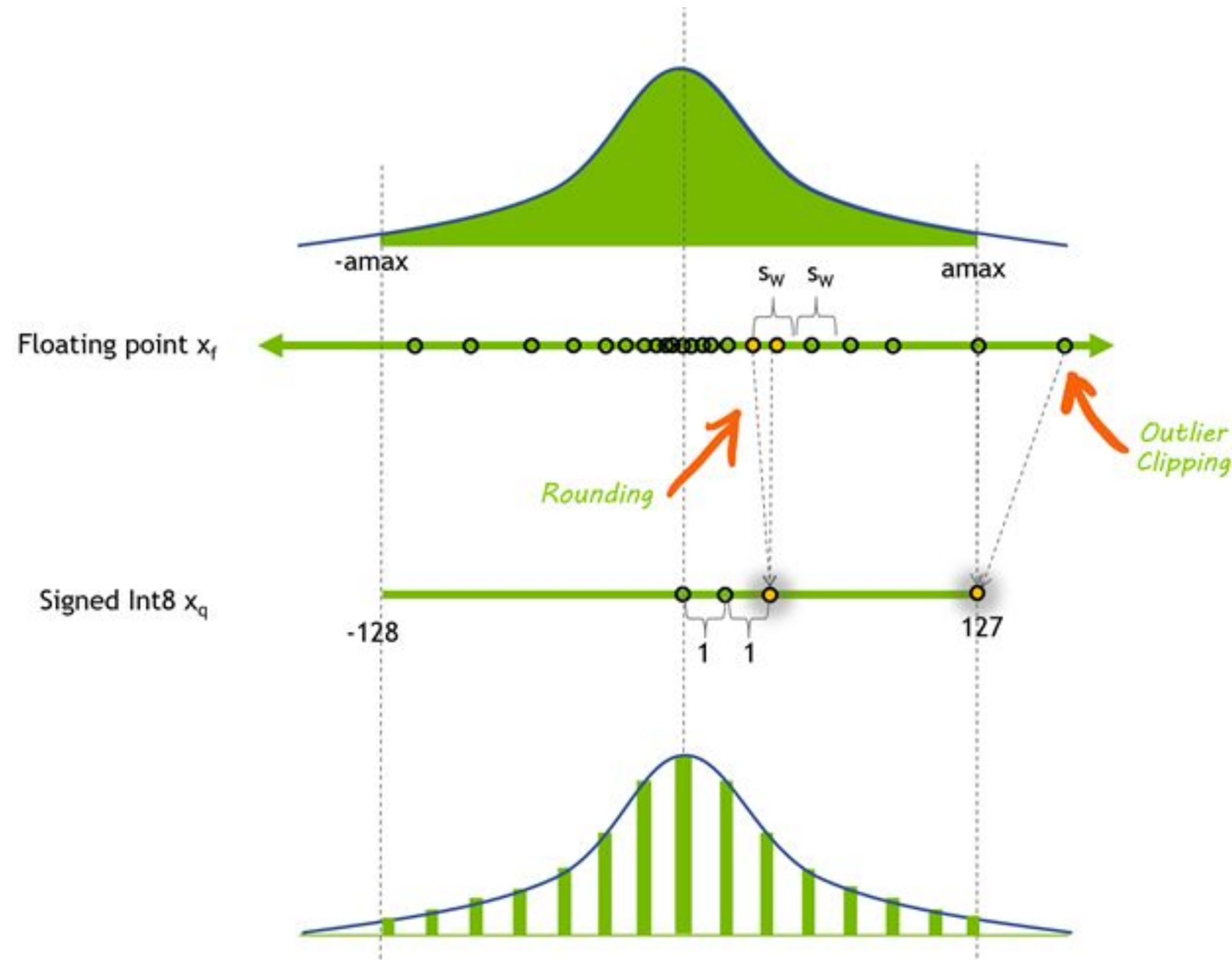


TRT's Static Quantization

- INT8: [-127, 128]
- x_f : FP32 Tensor
- x_q : INT8 Tensor

Symmetric: $x_q = \text{Clip}(\text{Round}(x_f / \text{scale}))$
, где clip - throw outliers (-128,127)

$$\text{scale} = 2 * \text{amax} / 256$$



Quantization Aware Training

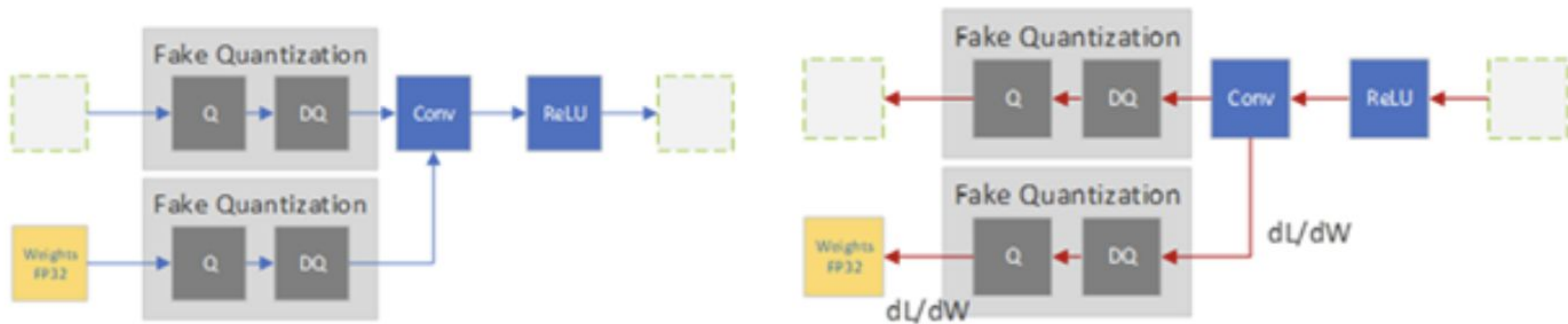


Figure 2. QAT fake-quantization operators in the training forward-pass (left) and backward-pass (right)

Extra links

- Accelerating Sparse Deep NN: <https://arxiv.org/pdf/2104.08378.pdf>
- Sparsity in INT8 for TRT:
<https://developer.nvidia.com/blog/sparsity-in-int8-training-workflow-and-best-practices-for-tensorrt-acceleration/>
- Q/DQ Layer-Placement Recommendations:
<https://docs.nvidia.com/deeplearning/tensorrt/developer-guide/index.html#qd-q-placement-recs>
- Dynamic shapes in TRT
<https://docs.nvidia.com/deeplearning/tensorrt/latest/inference-library/work-dynamic-shapes.html>
- Custom Layers in TRT
<https://docs.nvidia.com/deeplearning/tensorrt/latest/inference-library/extending-custom-layers.html>
- Deep Dive in TRT
<https://docs.nvidia.com/deeplearning/tensorrt/latest/architecture/how-trt-works.html>

Extra links

- TensorRT-LLM: <https://github.com/NVIDIA/TensorRT-LLM>
- ONNX-Docs: <https://onnx.ai/onnx/>

Спасибо за внимание!

Жду вопросов и обсуждений

